

Java Backend Architecture

Interview Series

Chapter 19.8

GCP Messaging & Data

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.8 – GCP Messaging & Data

Overview

Pub/Sub, Firestore, and BigQuery are GCP's primary messaging and data services for Java backends. Pub/Sub provides at-least-once global messaging; Firestore offers real-time document storage; BigQuery delivers serverless petabyte-scale analytics. Understanding message ordering, Firestore transactions, and BigQuery partitioning is required for GCP-focused Java architects.

Key Definitions

Term	Definition
Pub/Sub Topic	Messaging channel. Publishers send messages; subscriptions receive them.
Pub/Sub Subscription	Named subscriber to a topic. Pull (consumer polls) or Push (Pub/Sub POSTs to HTTP endpoint).
Pub/Sub Ordering Key	Ensures messages with same key delivered in order to same subscriber.
Dead-Letter Topic	Pub/Sub equivalent of DLQ. After maxDeliveryAttempts, message forwarded here.
Pub/Sub Snapshot/Seek	Save subscription state at a point in time. Seek to replay messages or skip ahead.
Firestore	Serverless document-collection NoSQL. Real-time listeners, ACID transactions, offline support.
Firestore Transaction	ACID read-modify-write across multiple documents. Serializable isolation.
BigQuery	Serverless column-oriented OLAP. SQL interface. Petabyte scale. Partitioning + clustering.
BigQuery ML	Run ML models (linear regression, logistic, k-means) directly in BigQuery SQL.
Pub/Sub Lite	Zonal, cheaper Pub/Sub variant with lower durability. Good for high-volume internal messaging.

Diagram 1: Pub/Sub Fan-out Pattern

```
Java Microservice (Publisher)
|
| publish: OrderEvent{orderId, status, userId}
v
Pub/Sub Topic: order-events
|
+-----+-----+
v         v         v
Sub:      Sub:      Sub:
email    invoice  analytics
push     pull     pull
|         |         |
v         v         v
HTTP      Java      BigQuery
Endpoint  Consumer  Streaming
(Cloud    (Spring    Insert
Run)     @PubSub)
```

Diagram 2: Firestore Document Structure

```
Collection: users
Document: user123
  name: "John Doe"
  email: "john@example.com"
  createdAt: 2024-01-15

Sub-collection: orders
  Document: order001
    status: "DELIVERED"
    total: 150.00
    items: [{productId: "p1", qty: 2}]

  Document: order002
    status: "PENDING"
    total: 89.99

Real-time listener: onSnapshot(query) fires on every change
```

Code Examples

Pub/Sub Publisher in Java (Spring):

```
// Spring Cloud GCP Pub/Sub
@Service
public class OrderEventPublisher {
    private final PubSubTemplate pubSubTemplate;

    public void publishOrderEvent(OrderEvent event) {
        String json = objectMapper.writeValueAsString(event);

        pubSubTemplate.publish("order-events", json,
            Map.of("eventType", event.getType(), // message attributes for filtering
                "userId", event.getUserId()));
    }
}

// Consumer (pull subscription)
@PubSubSubscriber(subscription = "order-events-invoice-sub",
    parallelPullCount = 2,
    maxOutstandingElementCount = 100)
public void processOrderEvent(
    @Payload OrderEvent event,
    PubsubMessage message,
    AckReplyConsumer ack) {
    invoiceService.generateInvoice(event);
    ack.ack(); // acknowledge: message deleted from subscription
}
```

Firestore ACID Transaction:

```
// Java Firestore transaction
Firestore db = FirestoreOptions.getDefaultInstance().getService();

ApiFuture<Void> transaction = db.runTransaction(t -> {
    // Read current inventory
    DocumentSnapshot product = t.get(db.collection("products").document(productId)).get();
    int currentStock = product.getLong("stock").intValue();

    if (currentStock < requestedQty) {
        throw new InsufficientStockException("Not enough stock");
    }

    // Decrement stock and create order atomically
    t.update(db.collection("products").document(productId),
        "stock", FieldValue.increment(-requestedQty));
    t.create(db.collection("orders").document(orderId),
        Map.of("status", "CONFIRMED", "qty", requestedQty));

    return null; // null = commit
});
transaction.get(); // throws on conflict (automatic retry by SDK)
```

BigQuery Streaming Insert from Java:

```
// BigQuery streaming insert
BigQuery bigquery = BigQueryOptions.getDefaultInstance().getService();
TableId tableId = TableId.of("java_analytics", "page_views");

InsertAllRequest insertRequest = InsertAllRequest.newBuilder(tableId)
    .addRow("row-" + event.getId(), Map.of(
        "user_id", event.getUserId(),
        "page", event.getPage(),
        "duration_ms", event.getDurationMs(),
        "timestamp", event.getTimestamp().toString()
    ))
    .build();
```

```
InsertAllResponse response = bigquery.insertAll(insertRequest);
if (response.hasErrors()) {
    log.error("BigQuery insert errors: {}", response.getInsertErrors());
}
```

Interview Q&A – GCP Messaging & Data

Q1. How does Pub/Sub message ordering work?

Standard Pub/Sub: at-least-once, no ordering guarantee. Ordering keys: messages with same key delivered in order to same subscriber. Enable on subscription: `enable_message_ordering=true`. Publisher sets `ordering_key` attribute. Limitation: all messages with same key go to same subscriber instance (reduces parallelism). Use case: user action events where per-user ordering matters (user#123's events processed in sequence).

Q2. What is the difference between Pub/Sub push and pull subscriptions?

Pull: consumer polls for messages (SubscriberStubSettings). Java app controls consumption rate, handles backpressure. Better for batch processing, variable rate. Push: Pub/Sub delivers to an HTTP endpoint (Cloud Run URL, HTTPS webhook). Java endpoint must be publicly accessible, return 2xx within deadline, handle retries. Better for event-driven Cloud Run services. Spring Cloud GCP: `@PubSubSubscriber` annotation uses pull internally.

Q3. How does Firestore differ from Cloud Datastore?

Firestore is the next generation of Datastore. Differences: stronger consistency model (all reads are strongly consistent, Datastore had eventual consistency on queries), real-time listeners (Datastore has none), collection/document model vs Kind/Entity, offline support for mobile. Firestore has two modes: Native Mode (new features) and Datastore Mode (Datastore API compatibility). New projects should use Firestore Native.

Q4. How does Firestore handle concurrent writes?

Firestore transactions use optimistic concurrency: transaction reads document version, on commit checks version unchanged. If changed by another transaction: automatic retry (up to 5 times). For high-contention documents (inventory counter): use `FieldValue.increment()` -- server-side atomic increment without transaction overhead. Distributed counters pattern: multiple shards with `FieldValue.increment`, aggregate on read.

Q5. What is BigQuery partitioning and why is it important?

Partitioning divides a table into segments by partition column. Date/time partitioning: each day is a separate partition. Queries filtering by date scan only relevant partitions -- reduces cost (BigQuery bills per byte scanned) and improves speed. Integer range partitioning: for non-time data. Clustering: further organises data within partitions by column values (`user_id`, `country`). Partition + cluster on frequently filtered columns reduces scanned data by 90%+.

Q6. When should you use Pub/Sub Lite instead of standard Pub/Sub?

Pub/Sub Lite: zonal (data in one zone, lower durability), 90% cheaper, throughput reserved (must provision capacity). Use for: high-volume internal messaging where cost matters and cross-zone durability is acceptable (application-level replication handles failures). Standard Pub/Sub: multi-zone durability, global topics across regions, auto-scaling. Use for: critical business events (order placed), cross-region messaging.

Q7. How does Pub/Sub dead-letter work?

After `maxDeliveryAttempts` (5-100) failed deliveries (NACK or deadline exceeded), Pub/Sub forwards message to dead-letter topic. Subscribe to DLT to alert or inspect failed messages. Use Pub/Sub Snapshot to replay DLT messages after fixing the consumer bug. Grant DLT publisher role to the source subscription's service account. Cloud Monitoring alert: `dead_letter_message_count > 0` triggers PagerDuty.

Q8. How do you stream events from Java to BigQuery for analytics?

Options: 1) BigQuery Streaming Insert: real-time, sub-second latency, \$0.01/200MB. 2) Pub/Sub -> Dataflow -> BigQuery: durable pipeline, handles backpressure, batch or streaming. Best for high-volume Java event streams. 3) Scheduled export: Java writes to GCS, BigQuery loads from GCS (batch, cheapest). For Java microservice event analytics: Pub/Sub -> Dataflow -> BigQuery is the recommended pattern.

Q9. What are Firestore security rules and how do they protect Java backends?

Firestore security rules (Firebase) control client SDK access (mobile/web) without backend involvement. Rules: allow read if `request.auth.uid == resource.data.userId`. For Java backends using Admin SDK: security rules are bypassed -- backend has full access. Rules protect direct client SDK access (mobile apps). Java Spring Boot using Firestore Admin SDK: implement access control at service layer, not rules.

Q10. How does BigQuery federated query work?

BigQuery can query external data without loading it: federated query over Cloud Storage (CSV/Parquet/Avro), Cloud Bigtable (read-only), Google Sheets, Cloud Spanner. `CREATE EXTERNAL TABLE OPTION (uris=['gs://bucket/data/*.parquet'],`

format='PARQUET'). SQL queries join BigQuery managed tables with external sources. Java use case: run analytics joining DynamoDB exports in S3/GCS with BigQuery transactional data without ETL pipeline.